

Sección 4: Seguridad con Keycloak

- Por qué un proyecto de arquitectura necesita seguridad
- Mapeo JPA profesional: @Table, @Column y referencia entre aggregates por ID
- Docker lo justo: imagen vs contenedor, dolor de docker run, Docker Compose como solución
- Keycloak con Docker Compose: realm, client, roles y usuarios
- Spring Boot como Resource Server: JWT y issuer-uri
- SecurityFilterChain con roles y converter de Keycloak (realm_access.roles)
- La seguridad vive en infraestructura, el dominio no sabe que Keycloak existe

38. ¿Por qué seguridad en un proyecto de arquitectura?

- **Keycloak** como Identity Provider (Docker Compose)
- Spring Boot como **Resource Server** (valida JWT)
- **SecurityFilterChain** con protección por roles
- Dos roles: **USER** y **ADMIN**
- Endpoints protegidos según quién opera



USER

Opera sus propias cuentas
Depositar, retirar, transferir
Consultar saldo y extractos



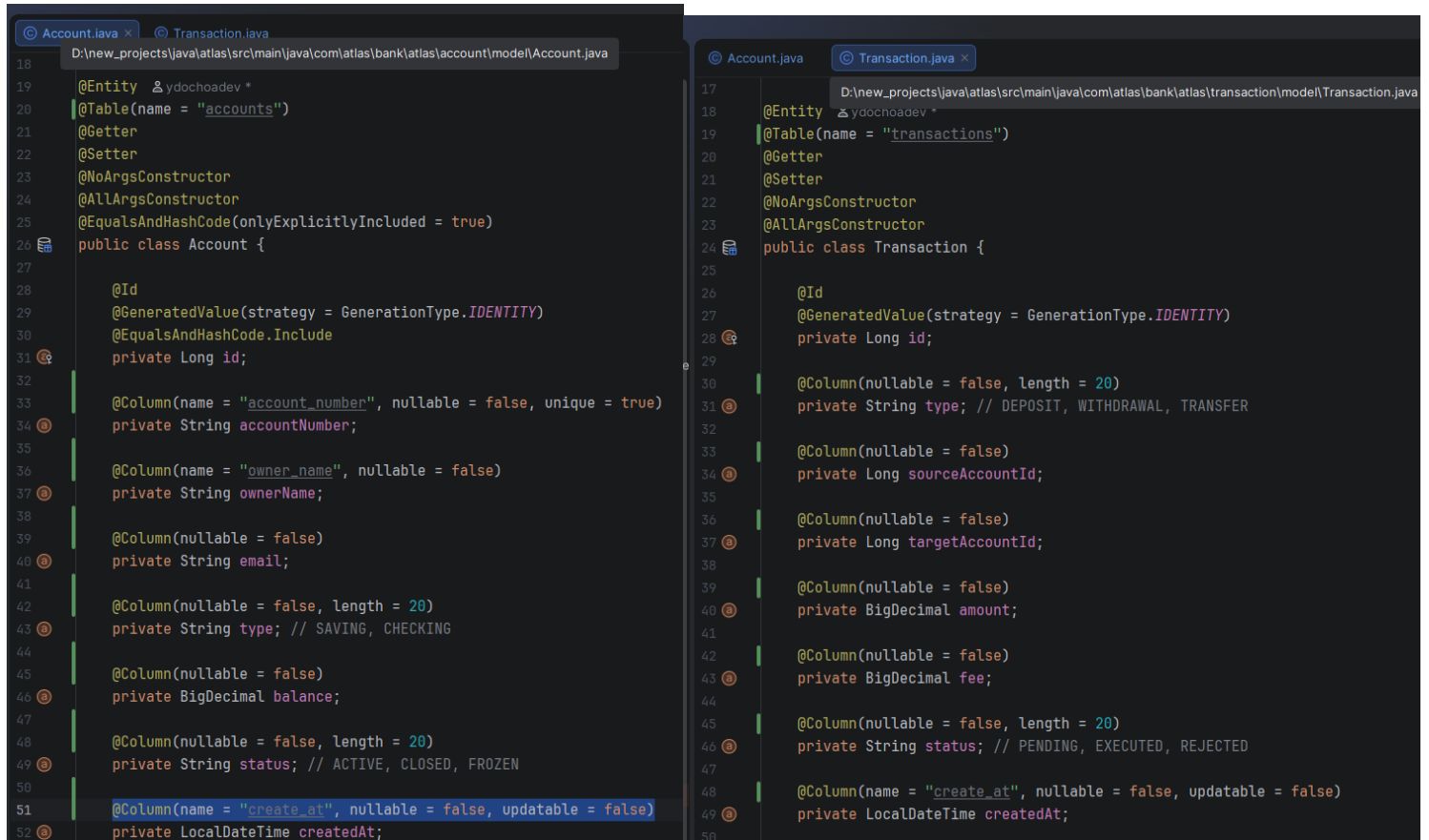
ADMIN

Todo lo de USER + gestión
Congelar y activar cuentas
Ver dashboard global

39. Mapeo JPA: @Table, @Column y referencia por Id

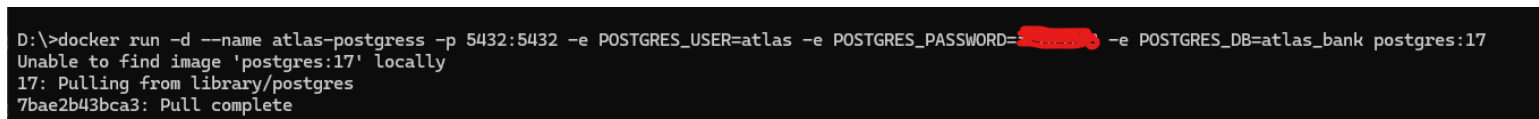
<https://www.objectdb.com/api/java/jpa/annotations>

➔ Refactorizando las entities:



```
Account.java | Transaction.java
D:\new_projects\java\atlas\src\main\java\com\atlas\bank\atlas\account\model\Account.java | D:\new_projects\java\atlas\src\main\java\com\atlas\bank\atlas\transaction\model\Transaction.java
18 | @Entity & ydochoadev * | 17 | @Entity & ydochoadev *
19 | @Table(name = "accounts") | 19 | @Table(name = "transactions")
20 | @Getter | 20 | @Getter
21 | @Setter | 21 | @Setter
22 | @NoArgsConstructor | 22 | @NoArgsConstructor
23 | @AllArgsConstructor | 23 | @AllArgsConstructor
24 | @EqualsAndHashCode(onlyExplicitlyIncluded = true) | 24 | @EqualsAndHashCode(onlyExplicitlyIncluded = true)
25 | public class Account { | 25 | public class Transaction {
26 | | 26 |
27 | | 27 |
28 | | 28 |
29 | @Id | 29 | @Id
30 | @GeneratedValue(strategy = GenerationType.IDENTITY) | 30 | @GeneratedValue(strategy = GenerationType.IDENTITY)
31 | @EqualsAndHashCode.Include | 31 | @EqualsAndHashCode.Include
32 | private Long id; | 32 | private Long id;
33 | | 33 |
34 | @Column(name = "account_number", nullable = false, unique = true) | 34 | @Column(nullable = false, length = 20)
35 | private String accountNumber; | 35 | private String type; // DEPOSIT, WITHDRAWAL, TRANSFER
36 | | 36 |
37 | @Column(name = "owner_name", nullable = false) | 37 | @Column(nullable = false)
38 | private String ownerName; | 38 | private Long sourceAccountId;
39 | | 39 |
40 | @Column(nullable = false) | 40 | @Column(nullable = false)
41 | private String email; | 41 | private Long targetAccountId;
42 | | 42 |
43 | @Column(nullable = false, length = 20) | 43 | @Column(nullable = false)
44 | private String type; // SAVING, CHECKING | 44 | private BigDecimal amount;
45 | | 45 |
46 | @Column(nullable = false) | 46 | @Column(nullable = false)
47 | private BigDecimal balance; | 47 | private BigDecimal fee;
48 | | 48 |
49 | @Column(nullable = false, length = 20) | 49 | @Column(nullable = false, length = 20)
50 | private String status; // ACTIVE, CLOSED, FROZEN | 50 | private String status; // PENDING, EXECUTED, REJECTED
51 | | 51 |
52 | @Column(name = "create_at", nullable = false, updatable = false) | 52 | @Column(name = "create_at", nullable = false, updatable = false)
53 | private LocalDateTime createdAt; | 53 | private LocalDateTime createdAt;
```

40. Docker: repasando conceptos



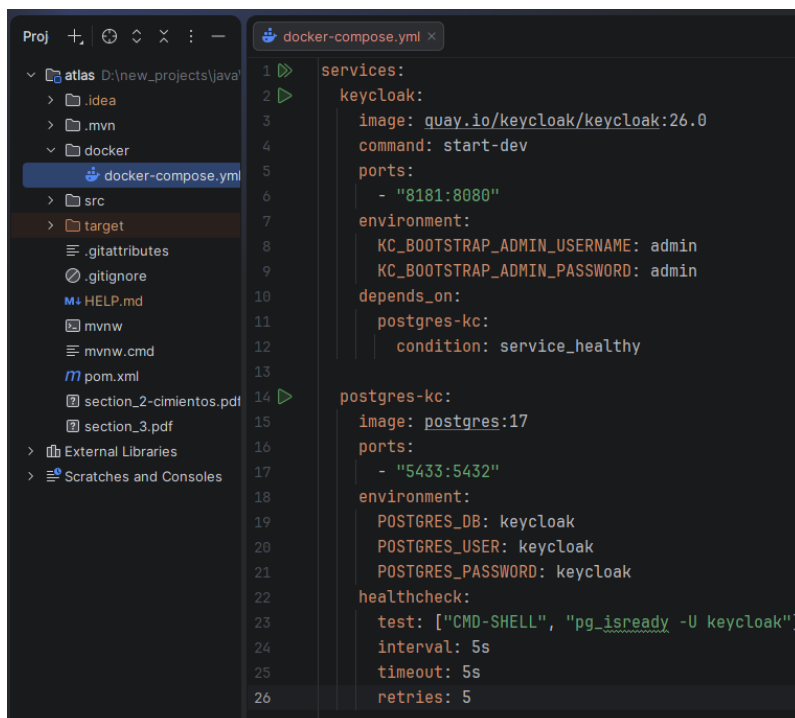
```
D:\>docker run -d --name atlas-postgress -p 5432:5432 -e POSTGRES_USER=atlas -e POSTGRES_PASSWORD=12345678 -e POSTGRES_DB=atlas_bank postgres:17
Unable to find image 'postgres:17' locally
17: Pulling from library/postgres
7bae2b43bca3: Pull complete
```

41. Docker compose: La opción

➔ ELIMINAR los contenedores creados.

42. Keycloak con Docker compose - parte 1 y 2

<https://gist.github.com/gchaldu/b9d039f1a651e175f4ae590263adac78>



```
Proj | docker-compose.yml
1 | services:
2 |   keycloak:
3 |     image: quay.io/keycloak/keycloak:26.0
4 |     command: start-dev
5 |     ports:
6 |       - "8181:8080"
7 |     environment:
8 |       KC_BOOTSTRAP_ADMIN_USERNAME: admin
9 |       KC_BOOTSTRAP_ADMIN_PASSWORD: admin
10 |     depends_on:
11 |       postgres-kc:
12 |         condition: service_healthy
13 |
14 |   postgres-kc:
15 |     image: postgres:17
16 |     ports:
17 |       - "5433:5432"
18 |     environment:
19 |       POSTGRES_DB: keycloak
20 |       POSTGRES_USER: keycloak
21 |       POSTGRES_PASSWORD: keycloak
22 |     healthcheck:
23 |       test: ["CMD-SHELL", "pg_isready -U keycloak"]
24 |       interval: 5s
25 |       timeout: 5s
26 |       retries: 5
```

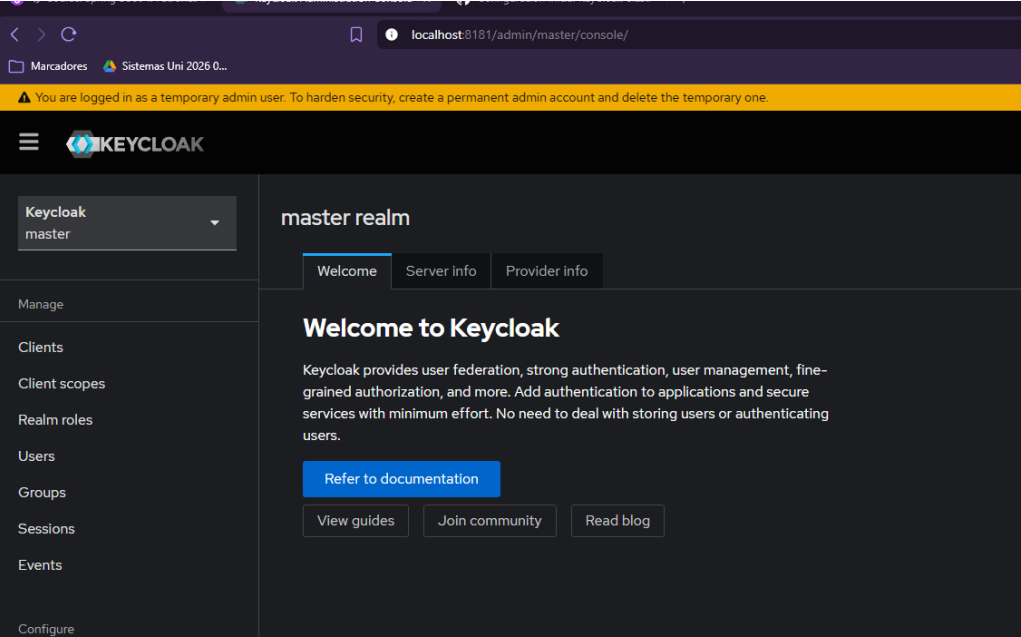
```
USER@DESKTOP-7UFJ8GM MINGW64 /d/new_projects/java/atlas/docker (main)
$ docker-compose up -d
[+] up 4/7
- Image quay.io/keycloak/keycloak:26.0 [■■■■■■■■] 155.1MB / 237.7MB Pulling
```

➔ Ir a keycloak:

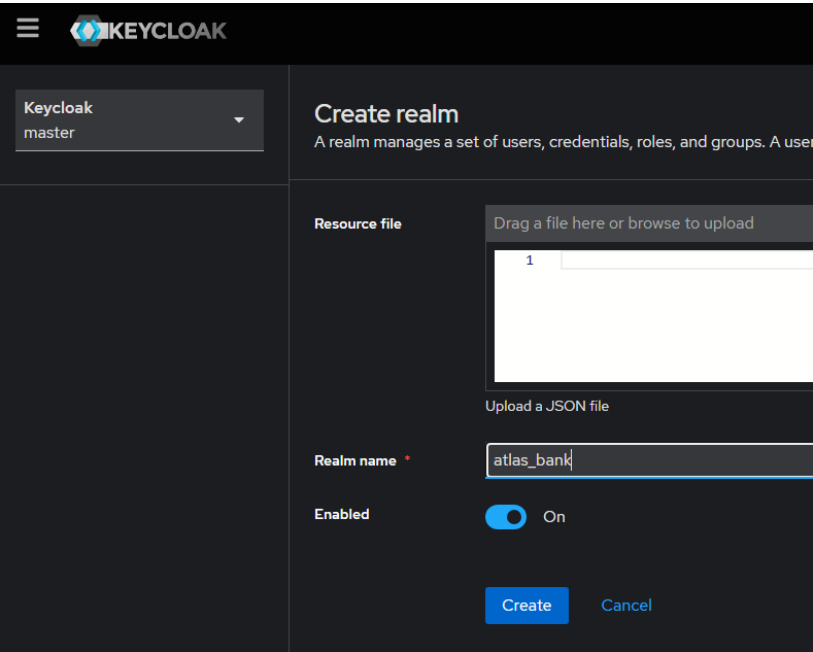
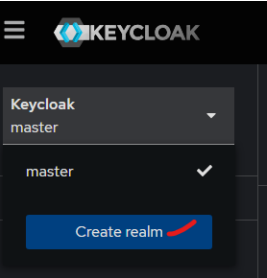
localhost:8181

user: admin

pass: admin



➔ Crear un realm (reino): Espacio aislado de seguridad.



➔ Crear un cliente: Representa la aplicación en springboot. Y keycloak requiere saber quién le está pidiendo tokens.

Create client

Clients are applications and services that can request authentication of a user.

1 General settings

2 Capability config

3 Login settings

Client type ⓘ

OpenID Connect

Client ID * ⓘ

atlas-bank-api

Name ⓘ

Description ⓘ

Always display in UI ⓘ

Off

1 General settings

2 Capability config

3 Login settings

Client authentication ⓘ

On

Authorization ⓘ

Off

Authentication flow

☒ Standard flow ⓘ

☒ Direct access grants ⓘ

☐ Implicit flow ⓘ

☐ Service accounts roles ⓘ

☐ OAuth 2.0 Device Authorization Grant ⓘ

☐ OIDC CIBA Grant ⓘ

- Standard flow: Para producción, cuando hay un usuario real: El usuario quiere entrar a la aplicación, esta lo redirige hacia keycloak, el usuario pone su contraseña directamente en keycloak, este lo valida y devuelve un token a la aplicación.
- Direct access grants: Permite que la aplicación reciba el usuario y la contraseña directamente. Pero este flujo toca la contraseña del usuario, la recibe y la transmite; la contraseña podría quedar en los logs; expone un endpoint para ataques de fuerza bruta. En PRODUCCIÓN SE DESACTIVA.
- Implicit flow: deprecado
- OAuth 2.0 Device Authorization Grant: Dispositivos sin teclado.

Create client

Clients are applications and services that can request authentication of a user.

1 General settings

2 Capability config

3 Login settings

Root URL ⓘ

Home URL ⓘ

Valid redirect URIs ⓘ

+ Add valid redirect URIs

Valid post logout redirect URIs ⓘ

+ Add valid post logout redirect URIs

Web origins ⓘ

+ Add web origins

➔ Crear roles:

Create role

Role name *

ROLE_USER

Description

Save

Cancel

Create role

Role name *

ROLE_ADMIN

Description

Save

Cancel

➔ Crear usuarios con sus roles:

Create user

Required user actions

Select action

Email verified

On

General

Username

cliente1

Email

cliente1@gmail.com

First name

First name

Last name

Last name

Groups

Join Groups

Create user

Required user actions

Select action

Email verified

On

General

Username

cliente2

Email

cliente2@gmail.com

First name

First name

Last name

Last name

Groups

Join Groups

** Crear password a cliente2:

Users > User details

cliente2

Details

Credentials

Role mapping

Groups

Consents

Identity provider links

Sessions

Set password for cliente2

Password

Password confirmation

Temporary

Off

Save

Cancel

** Asignar role de usuario:

cliente2

Details

Assign roles to cliente2

Filter by realm roles

Search by role name

Name

offline_access

ROLE_ADMIN

☒ROLE_USER

uma_authorization

Description

role_c

-

-

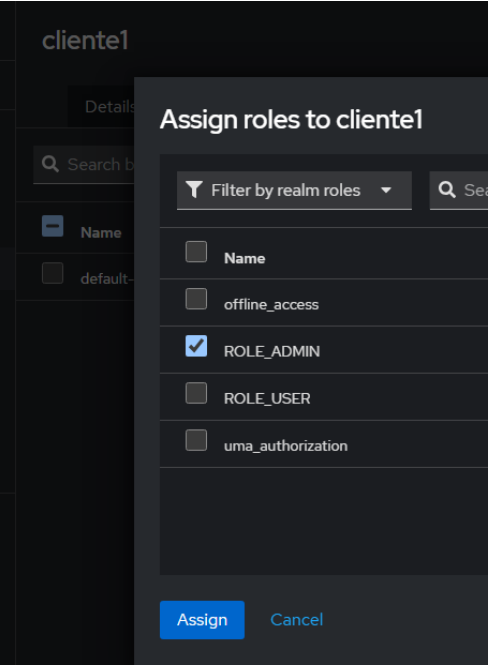
role_u

Assign

Cancel

➔ Cliente1 será admin:

** Crear contraseña y asignar role:



➔ Completar los datos: necesarios para generar token:

cliente1

Details

Credentials

Role mapping

Groups

Cons

ID *

5ea3ce2e-370d-4d9e-a093-68ed47d2dce

Created at *

9/13/2026, 4:32:36 PM

Required user actions

Select action

Email verified ?

On

General

Username *

cliente1

Email

cliente1@gmail.com

First name

Daniel

Last name

Ochoa

cliente2

Details

Credentials

Role mapping

ID *

3bc11015-193a-4269-80ec

Created at *

9/13/2026, 4:33:03 PM

Required user actions

Select action

Email verified ⓘ

☒ On

General

Username *

cliente2

Email

cliente2@gmail.com

First name

Lola

Last name

Perez

44. Spring boot como Resource Server

➔ En el modelo oauth2 hay 3 actores:

1 usuario

2 Authorization server ➔ keycloak

3 resource server ➔ api de spring boot. Token

Dependencia:

```

pom.xml (atlas) x
2   <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
34   <dependencies> Add Starters...
49       <artifactId>spring-boot-starter-webmvc</artifactId>
50   </dependency>
51   <!-- Agregando dependencia mapstruct -->
52   <dependency>
53       <groupId>org.mapstruct</groupId>
54       <artifactId>mapstruct</artifactId>
55       <version>${mapstruct.version}</version>
56   </dependency>
57   <!-- oauth2 -->
58   <dependency>
59       <groupId>org.springframework.boot</groupId>
60       <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
61   </dependency>
62
63   <dependency>
64       <groupId>com.h2database</groupId>
65       <artifactId>h2</artifactId>

```

** ¿Se necesita spring boot started security? NO, en oauth2 viene incluido.

Projec + | ⌕ | ↻ | ✕ | : | —

atlas D:\new_projects\java\at

idea

.mvn

docker

src

main

java

resources

static

templates

application.yaml

test

target

.gitattributes

.gitignore

HELP.md

mvnw

mvnw.cmd

pom.xml

section_2-cimientos.pdf

section_3.pdf

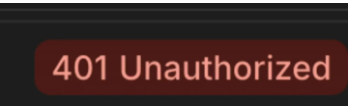
External Libraries

Scratches and Consoles

application.yaml x

```
1 spring:
2   application:
3     name: atlas
4
5   datasource:
6     url: jdbc:h2:mem:atlasbank
7     driver-class-name: org.h2.Driver
8     username: sa
9     password:
10
11   jpa:
12     database-platform: org.hibernate.dialect.H2Dialect
13     hibernate:
14       ddl-auto: create-drop
15       show-sql: true
16
17   security:
18     oauth2:
19       resourceserver:
20         jwt:
21           issuer-uri: http://localhost:8181/realms/atlas_bank
22
23   h2:
24     console:
25       enabled: true
```

➔ Levantar el proyecto y al ejecutar un endpoint, no funciona, sale error 401:



Por defecto, todos los endpoints se protegen automáticamente.

atlas_bank

Manage

Clients

Client scopes

Realm roles

Users

Groups

Sessions

Events

Configure

Realm settings

Authentication

Clients

Clients are applications and services that can

Clients list

Initial access token

Create

Search for client

→

Client ID	Name
account	client
account-console	client
admin-cli	client
atlas-bank-api	—
broker	client
realm-management	client
security-admin-console	client

Clients > Client details

atlas-bank-api OpenID Connect

Clients are applications and services that can request authentication of a user.

Settings

Keys

Credentials

Roles

Client scopes

Sessions

Advanced

Client Authenticator

Client Id and Secret

Save

Client Secret

.....

👁

🗑

Params
Body
Headers
Auth
Vars
Script
Assert
Tests
Docs
File
»
OAuth 2.0

Token Type: Bearer
Scope: profile email

Configuration

Access Token URL
http://localhost:8181/realms/atlas_bank/protocol/openid-conne

Username
cliente1

Password

Client ID
atlas-bank-api

Client Secret

Scope

Add Credentials to
Request Body

Token

Token Source
Access Token

Token ID
credentials

Add token to
Headers

Header Prefix
Bearer

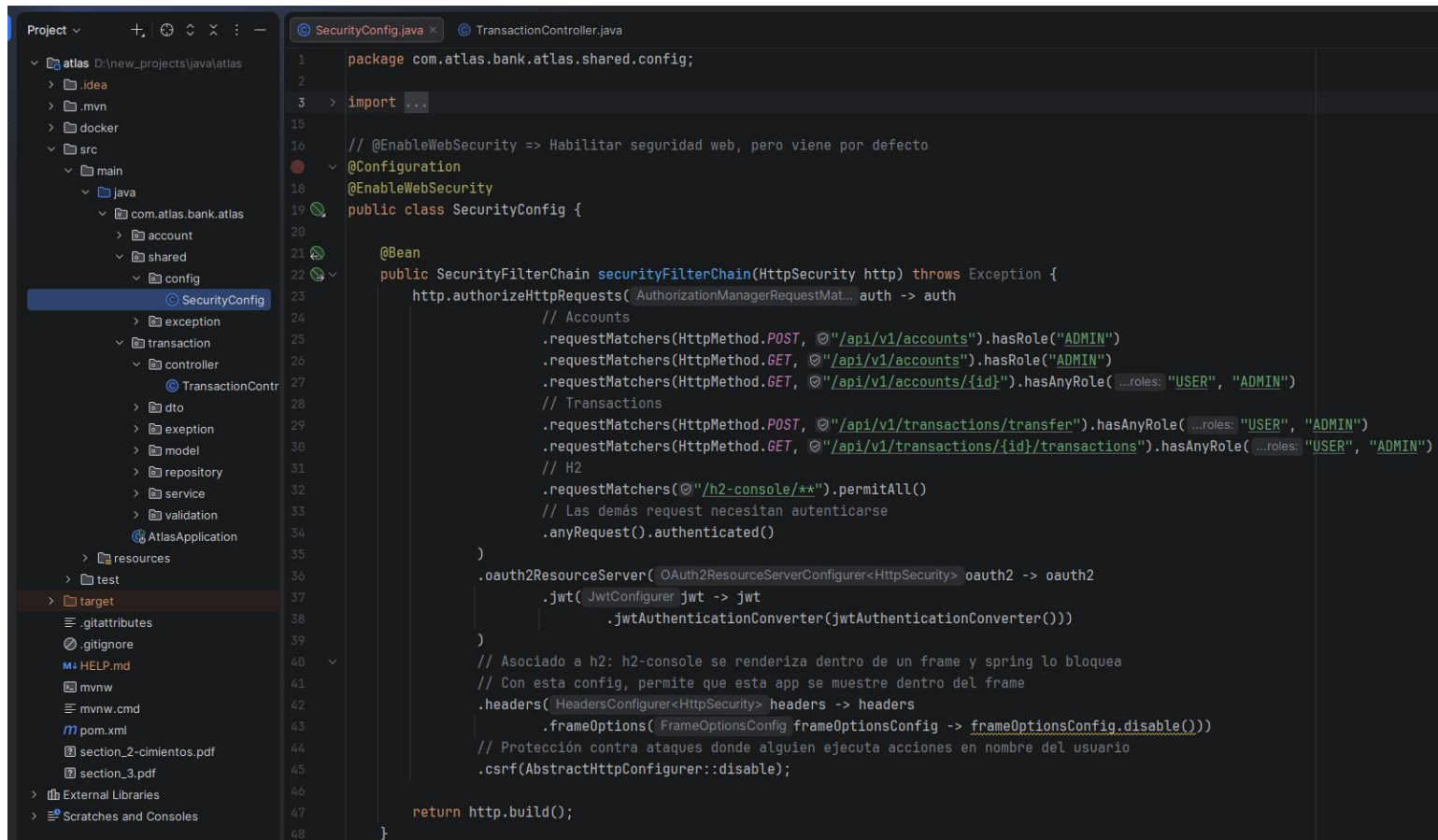
45. SecurityFilterChain: protección basada en rol 1 y 2

→ Role user solo pueda transferir y consultar, NO crear cuantas.

→ Role admin podrá crear cuentas.

→ Spring security evalúa las reglas en ORDEN: Cuando encuentra la primera regla que coincide con la request, deja de evaluar y la aplica

** .oauth2ResourceServer(...) → Valida un jwt. Se le pase un converter que se debe crear.



```
// Conversor personalizado: Los roles están en formato keycloak y ese formato no se puede usar
// Por ello se crea el converter para transformarlo a SimpleGrantedAuthority
private JwtAuthenticationConverter jwtAuthenticationConverter() { 1 usage
    JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
    // Sacar los roles para poder convertirlos
    converter.setJwtGrantedAuthoritiesConverter( Jwt jwt -> {
        var realmAccess = jwt.getClaimAsMap("realm_access");
        if (realmAccess == null || realmAccess.get("roles") == null) {
            return List.of(); // Si no hay roles, no se asigna ninguno
        }
        var roles = (List<String>) realmAccess.get("roles");

        return roles.stream() Stream<String>
            .map(SimpleGrantedAuthority::new) Stream<SimpleGrantedAuthority>
            .collect(Collectors.toList());
    });

    return converter;
}
```

47. Probando la seguridad: USER y ADMIN

48. Seguridad como infraestructura, no como dominio

➔ "El dominio no sabe que Keycloak existe."

Responsabilidad	Ubicación	Toca el dominio?
Validar tokens JWT	SecurityConfig	✗ No
Proteger endpoints por rol	SecurityConfig	✗ No
Extraer roles del token	JwtAuthenticationConverter	✗ No
Validar datos del request	Bean Validation (DTOs)	✗ No
Validar reglas de negocio	Services / Entities	✓ Sí

¿Por qué importa?

- Si cambiamos Keycloak por otro provider, el **dominio no cambia**
- Si los endpoints cambian de ruta, los **services no se enteran**
- Los tests de dominio **no necesitan** Spring Security
- Cuando lleguemos a **Hexagonal**, SecurityConfig será un adaptador de infraestructura